

CPTS 360 PA 1: Cache Memory Simulation

Name: Isaiah Foster

WSU ID: 11796849

1. Overview

The simulator replays a valgrind memory trace against a configurable set-associative cache with LRU replacement and reports the number of hits, misses, and evictions. It accepts the cache parameters as `-s` (set index bits), `-E` (lines per set), and `-b` (block offset bits), so the cache has $S = 2^s$ sets of E lines each. The code is split into two parts:

- `cache.h / cache.c`: the cache data structure and the access/replacement logic, independent of trace format or output.
- `cachesim.c`: command-line parsing, trace file parsing, verbose output, and the call to `print_summary`.

The `Makefile` was updated to build both source files, and `test.sh` runs the four reference cases and diffs verbose output against the provided samples.

2. Data Structures

Each cache line is a `cache_line_t` holding a valid bit, a 64-bit tag, and a `last_used` timestamp. Block data is never stored, since only hit/miss behavior matters. The cache (`cache_t`) stores s , E , b , the running hit/miss/eviction counters, a global access clock, and a single array of $S \times E$ lines allocated with `calloc` (so every line starts as 0). Set i occupies lines iE through $iE + E - 1$. Using one flat allocation instead of an array of per-set arrays keeps allocation and cleanup to a single call each and keeps a set's lines contiguous in memory.

`cache_create` rejects geometries whose line count would overflow `size_t` and returns NULL if allocation fails; `cache_free` releases both allocations.

3. Address Decomposition

For an address a :

$$\text{set} = (a \gg b) \& (2^s - 1), \quad \text{tag} = a \gg (s + b)$$

The block offset is discarded because accesses are assumed never to cross a block boundary. In C, shifting a 64-bit value by 64 or more is undefined behavior, so when $b = 64$ or $s + b = 64$ the set index or tag is set to 0 explicitly.

4. Access and LRU Replacement

`cache_access` increments the global clock and makes one pass over the E lines in the selected set:

1. If a line is valid with a matching tag, it is a **hit**: the line's `last_used` is set to the current clock and `ACCESS_HIT` is returned.
2. During the same pass, a victim is tracked: the first invalid line if one exists, otherwise the valid line with the smallest `last_used` (the least recently used).
3. If no line matched, it is a **miss**. If the victim is valid, its contents are evicted and `ACCESS_MISS_EVICT` is returned; otherwise `ACCESS_MISS`. The victim is then filled with the new tag and the current timestamp.

A monotonically increasing counter is used instead of reordering lines on each access. This gives exact LRU ordering in $O(E)$ time per access, and because the counter is 64-bit it cannot overflow on any realistic trace. The function updates the counters itself and returns the outcome so the caller can print verbose output.

5. Trace Parsing

The trace is read one line at a time with `fgets`. Lines that do not begin with a space are instruction loads (I) and are skipped, as are any lines that `sscanf(" %c %llx,%d")` cannot parse or whose operation is not L, S, or M. The access size is read but ignored. Loads and stores each perform one cache access. A modify (M) performs two accesses to the same address (a load followed by a store), so its second access is always a hit. In verbose mode the operation, address, and size are printed, then `hit`, `miss`, or `miss eviction` for each access, matching the format of the provided sample outputs exactly.

6. Command-Line Handling

Arguments are parsed with `getopt` using the option string `"s:E:b:t:vh"`. Numeric values are converted with `strtol` and rejected if they are non-numeric, negative, or out of range. `-h` prints usage and exits with status 0. An unknown option, a missing required argument, $E < 1$, $s \geq 64$, or $s + b > 64$ prints usage and exits with status 1. The provided `print_usage` was changed to no longer call `exit(0)` itself, so the caller chooses the exit status. A trace file that cannot be opened is reported with `perror`.

7. Testing

The simulator builds with the provided flags (`-Wall -Werror -std=c99`) without warnings and produces the expected results for all four reference cases:

(s, E, b)	Hits	Misses	Evictions	Trace
(1, 1, 1)	9	8	6	trace01.dat
(4, 2, 4)	4	5	2	trace02.dat
(2, 1, 4)	2	3	1	trace03.dat
(5, 1, 5)	265189	21775	21743	trace04.dat

Verbose output for trace01 and trace02 matches the files in `traces/output/` byte for byte, and the direct-mapped (4, 1, 4) example from the assignment handout also matches. Additional geometries (fully associative $s = 0$, highly associative caches, $b = 64$) and invalid arguments were tested.